

Red Hat Lightspeed 1-latest

Getting started with MCP server for Red Hat Lightspeed

Last Updated: 2026-09-16

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document describes how to install and configure the MCP server for Red Hat Lightspeed server for use with Red Hat Lightspeed and an LLM-based AI agent.

Table of Contents

CHAPTER 1. ABOUT MCP SERVER FOR RED HAT LIGHTSPEED 3
1.1. DEVELOPER PREVIEW NOTICE 3
1.2. ABOUT MCP SERVER FOR RED HAT LIGHTSPEED 3
1.3. SUPPORTED RED HAT LIGHTSPEED SERVICES 3

CHAPTER 2. CONFIGURE THE MCP SERVER FOR RED HAT LIGHTSPEED SERVER 5
2.1. CREATE AND CONFIGURE A SERVICE ACCOUNT 5
2.2. INSTALL AND CONFIGURE A CONTAINER RUNTIME 6
2.3. INSTALL AND CONFIGURE YOUR MCP CLIENT 7
2.4. ABOUT RED-HAT-LIGHTSPEED-MCP TOOLSETS 7
2.4.1. How toolsets work 7
2.5. CONFIGURING ADDITIONAL OPTIONS FOR THE MCP SERVER 8
2.6. STOP AND RESTART THE MCP SERVER FROM THE COMMAND LINE 10
2.7. AVAILABLE OPTIONS FOR RED-HAT-LIGHTSPEED-MCP 11
2.7.1. Server connection options 11

CHAPTER 3. USE CASES AND EXAMPLES 14
3.1. ADDITIONAL RESOURCES 14
3.2. USE CASES AND EXAMPLES 14
3.3. SHARE YOUR FEEDBACK 14

CHAPTER 1. ABOUT MCP SERVER FOR RED HAT LIGHTSPEED

MCP server for Red Hat Lightspeed (**red-hat-lightspeed-mcp**) is a lightweight, self-hosted Model Context Protocol (MCP) server that you can use to interact with Red Hat Lightspeed.

Additional resources

- Developer Preview Support Scope.

1.1. DEVELOPER PREVIEW NOTICE

MCP server for Red Hat Lightspeed is Developer Preview software only.



IMPORTANT

MCP server for Red Hat Lightspeed is Developer Preview software only. Developer Preview software is not supported by Red Hat in any way and is not functionally complete or production-ready. Do not use Developer Preview software for production or business-critical workloads. Developer Preview software provides early access to upcoming product software in advance of its possible inclusion in a Red Hat product offering. Customers can use this software to test functionality and provide feedback during the development process. This software might not have any documentation, is subject to change or removal at any time, and has received limited testing. Red Hat might provide ways to submit feedback on Developer Preview software without an associated SLA.

Additional resources

- Developer Preview Support Scope

1.2. ABOUT MCP SERVER FOR RED HAT LIGHTSPEED

With MCP server for Red Hat Lightspeed, you can use simple language to get answers about your Red Hat Lightspeed environment without manually searching in the Red Hat Lightspeed UI. To communicate with Red Hat Lightspeed APIs by using natural language prompts, connect an LLM-based agent such as Visual Studio Code, Cursor, or other MCP-compatible AI tools to MCP server for Red Hat Lightspeed.

For example, suppose that you receive a critical alert for a CVE that affects systems in your environment. Instead of searching through multiple dashboards and documentation for information about the alert and the CVE, you can enable **--all-tools** and then use an LLM-based agent to ask MCP server for Red Hat Lightspeed, "Which systems are affected by this CVE?"

MCP server for Red Hat Lightspeed allows the LLM to access your Red Hat Lightspeed data and instantly provides the diagnostic information and a specific remediation playbook for that CVE. You can then take immediate action to secure your systems and data and keep your operations running smoothly.

1.3. SUPPORTED RED HAT LIGHTSPEED SERVICES

MCP server for Red Hat Lightspeed supports the following Red Hat Lightspeed services.

- Advisor

- Hosted image builder
- Inventory
- Digital roadmaps and planning for RHEL
- Remediations
- Vulnerability
- RHSM
- Content Sources

Additional resources

- What is Model Context Protocol?
- Video: Using AI to query Red Hat Lightspeed

CHAPTER 2. CONFIGURE THE MCP SERVER FOR RED HAT LIGHTSPEED SERVER

Before you can use MCP server for Red Hat Lightspeed, you must create or configure settings in the Hybrid Cloud Console and install the LLM-based agent you want to use.

2.1. CREATE AND CONFIGURE A SERVICE ACCOUNT

By default, service accounts do not automatically have access to Red Hat Lightspeed services. You must configure specific roles for your service account, depending on the services you want to use with MCP server for Red Hat Lightspeed.

You configure the roles on the **Add roles** page of the **Create group** wizard.

The following roles are required for the Red Hat Lightspeed service you want to use.

- Advisor: RHEL Advisor viewer
- Inventory: Inventory Hosts viewer
- Vulnerability: Vulnerability viewer and Inventory Hosts viewer
- Remediation: Remediations user

Prerequisites

- You are logged into the Hybrid Cloud Console as an Organization Administrator with the User Access role.

Procedure

1. From the Red Hat Hybrid Cloud Console, click the Settings icon (⚙) and click **Service Accounts**.
2. Click **Create Service Account** The **Create a Service account**dialog box displays.
3. Type the name for your service account and an optional short description, and then click **Create**. The **Credentials successfully generated** page displays.
4. Copy and save your client ID (**LIGHTSPEED_CLIENT_ID**) and client secret (**LIGHTSPEED_CLIENT_SECRET**), and then click the checkbox to acknowledge that you have saved them.
5. Click **Close** to close the page.
6. Click the Settings icon (⚙) and select **User Access**.
7. From the **User Access** menu on the left side of the window, select **Groups**.
8. Click **Create group**. The Create group wizard displays.
9. Type a name for your group.
   Give your group a recognizable name. For example, you could name the group **mcp-service-accounts**.

10. **Optional:** Add a brief description of the group.
11. Click **Next**. The **Add roles** page displays.
12. Add the required roles for your group (for example, RHEL Advisor viewer, Inventory Hosts viewer, and so on) and click **Next**. The **Add service accounts** page displays.
13. Add your service account to this group and click **Next**. The **Review details** page displays.
14. Click **Submit**.

Additional resources

- Creating and managing a service account
- Service account permissions setup

2.2. INSTALL AND CONFIGURE A CONTAINER RUNTIME

The easiest way to deploy the **insights-mcp** server is by using a container runtime, such as Podman or Docker.

===

- The following examples use Podman.
- If you use macOS, you might need to explicitly set the path to Podman after installation. Use the location where your Podman executable is installed: **/usr/local/bin/podman**, or **/opt/homebrew/bin/podman**.
- To find the correct path, type **which podman** at the shell prompt. ===

Prerequisites

- You are logged into your system as root.

Procedure

1. Open a terminal window to install Podman from the command line.
2. To install Podman on Fedora/RHEL/CentOS, type this command:

```
sudo dnf install podman
```

On macOS, either install Podman Desktop, or type this command to use Homebrew to install Podman:

```
brew install podman
```

Additional resources

- Install and initialize Podman Desktop and Podman
- Podman desktop

- Red Hat build of Podman desktop

2.3. INSTALL AND CONFIGURE YOUR MCP CLIENT

You can use the README file to install and configure your MCP client. MCP server for Red Hat Lightspeed has been tested with Visual Studio Code, Cursor, Gemini CLI, Claude Code, and Claude Desktop clients.

The README file includes one-click installation for Visual Studio Code and Cursor, and manual STDIO and streaming HTTP installation instructions for other clients. For Claude Desktop, the repository includes a downloadable extension file and installation instructions.

===

- The following examples use Visual Studio Code as the MCP client.
- If you start the MCP server locally with Podman or Docker, make sure that the container is not exposed to the internet. Your MCP client has access to your Client ID and Client Secret. If you connect to the MCP server from a different machine, the Client ID and Client Secret are transferred to the MCP server. This could result in a leak of the ID and Secret. If you are in doubt, disable or remove the **LIGHTSPEED_CLIENT_ID** and **LIGHTSPEED_CLIENT_SECRET** from your account after you are done using the MCP server. ===

Procedure

1. Use a web browser to navigate to Visual Studio and install the Visual Studio Code client on your system.
2. After Visual Studio Code is installed on your system, navigate to the README file of the MCP server for Red Hat Lightspeed GitHub repository and click **VS Code Install Insights MCP**.
3. When the installation process prompts you, enter the client ID and client secret that you created and saved when you set up your service account.

Additional resources

- README file
- Create and configure a service account

2.4. ABOUT RED-HAT-LIGHTSPEED-MCP TOOLSETS

Toolsets contain a list of operations for the corresponding Red Hat Lightspeed service. You request these operations in the LLM client that you installed, such as Visual Studio Code or Cursor.

Examples of toolsets include **inventory** or **vulnerability**. These toolsets contain tools like **get_cve** and **find_host_by_name**. The Available toolsets README file in the MCP server for Red Hat Lightspeed GitHub repository lists all of the currently available toolsets.

2.4.1. How toolsets work

Suppose you want to ask Red Hat Lightspeed about the CVEs that affect systems in your environment:

- When you issue a request from your LLM client (for example, **Which CVEs are affecting my system?**), the client selects the relevant tool (for example, **get_cve_systems**) .
- The LLM converts your request into a structured API call and passes it to the MCP server.
- The MCP server then passes the API call to the Red Hat Lightspeed vulnerability service, and receives a response in the form of a block of JSON code.
- The MCP server passes the JSON code back to the LLM client, which interprets the results and formats them into a clear, readable response. In this case, the response contains a list of the CVEs that affect your system.

Some toolsets are read-only by default. This means that the operations that they perform only involve reading information from Red Hat Lightspeed. Read-only toolsets are always enabled in the MCP server for Red Hat Lightspeed server.

Some toolsets perform read/write operations. The operations that these toolsets perform involve reading information from Red Hat Lightspeed and writing data or executing operations. **create_blueprint** is an example of a read/write tool. In the Available toolsets README file, the read/write toolsets are identified by (**rw**). Read/write toolsets are not enabled by default in the MCP server for Red Hat Lightspeed server. If you want to enable read/write operations, use the **--all-tools** option when you start the MCP server.



NOTE

RBAC permissions can also restrict access to tools. Make sure that your service account contains the correct permissions for the services and roles that you plan to use.

You might decide to use only specific toolsets for certain Red Hat Lightspeed services, or you might decide to add a service to your current configuration.

Additional resources

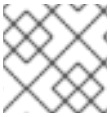
- Creating and managing a service account
- **red-hat-lightspeed-mcp** repository
- Available toolsets README file
- Configuring additional options for the MCP server

2.5. CONFIGURING ADDITIONAL OPTIONS FOR THE MCP SERVER

If you used one-click installation to install the MCP server, the LLM client built the server configuration in the background, and then automatically started the server in default (read-only) mode. If you manually installed and configured the server, you created the **mcp.json** configuration file before starting the server. You change or configure options for the MCP server in the **mcp.json** file.

There are two ways to configure additional options for the MCP server:

- Edit the **mcp.json** configuration file in a separate window, and then open a new chat window.
- Manually stop the MCP server, configure the options, and then restart the server.



NOTE

If you are using Gemini, the configuration file is called **settings.json**.

You can use the command line to manually stop and start the server. For information about how to use the command line, see Stop and restart the MCP server from the command line . The MCP server for Red Hat Lightspeed server README file also includes instructions about how to manually configure, start, and stop the server.

If you want to enable the read/write tools, edit the **mcp.json** file to add the **--all-tools** option. Likewise, if you manually installed and configured the MCP server and want to use the read/write tools, you must edit **mcp.json** to add the option.

The location of the MCP server configuration file on your system depends on the LLM client you are using. For example, if you are using Visual Studio Code, the file is located at **.vscode/mcp.json**. If you are using Cursor, the file is located at **~/.cursor/mcp.json**. To find the location of **mcp.json** for your LLM client, see the MCP server for Red Hat Lightspeed server README file .



NOTE

The following example uses Visual Studio Code.

Procedure

1. Use an editor to open the configuration file located at **`.vscode/mcp.json**.
2. Locate the **args** section inside the **red-hat-lightspeed-mcp** server definition.
3. Append **--all-tools** to the very end of the **args** section after the container image name. For example:

```
{
  "inputs": [
    {
      "id": "lightspeed_client_id",
      "type": "promptString",
      "description": "Enter the Red Hat Lightspeed Client ID",
      "default": "",
      "password": true
    },
    {
      "id": "lightspeed_client_secret",
      "type": "promptString",
      "description": "Enter the Red Hat Lightspeed Client Secret",
      "default": "",
      "password": true
    }
  ],
  "servers": {
    "lightspeed-mcp": {
      "type": "stdio",
      "command": "podman",
      "args": [
        "run",
        "--env", "LIGHTSPEED_CLIENT_ID",
```

```
        "--env", "LIGHTSPEED_CLIENT_SECRET",
        "--interactive",
        "--rm",
        "ghcr.io/redhatinsights/red-hat-lightspeed-mcp:latest",
        "--all-tools"
      ],
      "env": {
        "LIGHTSPEED_CLIENT_ID": "${input:lightspeed_client_id}",
        "LIGHTSPEED_CLIENT_SECRET": "${input:lightspeed_client_secret}"
      }
    }
  }
}
```

4. Save the **mcp.json** file.

5. Restart your LLM client or open a new chat session. The LLM client reads the updated **mcp.json** file and launches a fresh instance of the container with the write tools enabled.

Additional resources

- **red-hat-lightspeed-mcp** README file
- Available toolsets README file
- **red-hat-lightspeed-mcp** command usage README file
- Stop and restart the MCP server from the command line

2.6. STOP AND RESTART THE MCP SERVER FROM THE COMMAND LINE

Use the command line to manually stop and restart the MCP server. You might want to stop the MCP server to configure additional options or to update the **mcp.json** file.



NOTE

- The following example commands use Podman.
- If the procedure does not work as expected, make sure Podman is running and then try restarting your AI client.

Prerequisites

- You have Podman or Docker installed.
- You have installed the MCP server.

Procedure

1. To stop the MCP server, you must stop the container. If you are using Podman, enter the following command:

```
% podman ps
```

2. Look for the container that contains the MCP server. Copy the container ID.
3. Enter the following command to stop the container:

```
% podman stop <container_id>
```

   Replace **<container_id>** with the container ID that you copied from Podman.

4. To restart the container and the MCP server, enter the following command:

```
% podman run --net host --rm ghcr.io/redhatinsights/red-hat-lightspeed-mcp:latest http
```

2.7. AVAILABLE OPTIONS FOR RED-HAT-LIGHTSPEED-MCP

You can use the command line to configure multiple options for the MCP server for Red Hat Lightspeed server.

The **red-hat-lightspeed-mcp** command uses the following syntax:

```
lightspeed-mcp [-h] [--debug] [--toolset TOOLSET]
               [--toolset-help] [--readonly]|[--all-tools]
               {stdio|sse|http}
```

Option	Description
-h or **--help**	Show the help message for lightspeed-mcp, and then exit
--debug	Enable debug logging
--toolset TOOLSET	Comma-separated list of the toolset or toolsets you want to use. Options: **all**, **image-builder**, **rhsm**, **vulnerability**, **remediations**, **advisor**, **inventory**, **content-sources**, **rbac**, **planning** (Default: **all**)
--toolset-help	Show details for all the toolsets
--readonly	Only register read-only tools (default)
--all-tools	Register all tools, including read/write tools

2.7.1. Server connection options

The STDIO, SSE, and HTTP options for **red-hat-lightspeed-mcp** are different transport mechanisms for the Model Context Protocol (MCP). They determine how the LLM client (such as Visual Studio Code, Claude Desktop, or another AI tool) talks to the server (the **red-hat-lightspeed-mcp** code).

Option	Description
STDIO (Standard Input/Output)	This is the most common way to run the MCP server locally on your own system. You typically choose STDIO if you want to run **red-hat-lightspeed-mcp** on your laptop to analyze your own Red Hat Lightspeed data. Choose HTTP or SSE if you are hosting the MCP server in a central location for multiple people or tools to access using a URL. When you start your LLM client, it launches the MCP server as a child process on your computer. The client and server communicate to each other through the command line (Standard Input and Standard Output). This configuration does not require network ports because both client and server are processes on the same system. This option is best suited for local development and personal use. It runs very quickly because there is no network overhead, and it is secure because the server is not exposed to the internet. However, it only works if the AI client and the server are on the same machine.
SSE (Server-Sent Events)	SSE is a web-based transport that allows the server to push data to the client. The client and server can be on different systems. The client opens a persistent connection to a web URL. The server stays connected and sends updates or responses down that same line whenever they are ready. This option works best in remote setups where the AI client and the MCP server are on different systems, or even different networks. Since SSE is network-based, you must provide a URL (such as http://localhost:9000/sse) for the client to connect to the server.
HTTP (or Streamable HTTP)	Streamable HTTP is the modern standard that combines standard HTTP requests with SSE-like streaming capabilities. The client sends a specific web service request (POST) with a command, and the server sends back a single response. This option works best for cloud deployments, scalable services, or situations where you want to put the MCP server behind a load balancer or a standard web proxy.

Additional resources

- Available toolsets README
- lightspeed-mcp usage README

CHAPTER 3. USE CASES AND EXAMPLES

You can adapt the use cases and examples in this section and incorporate them into your Red Hat Lightspeed workflow. The sample prompts are available to copy and paste, or you can type your own natural-language prompts.

3.1. ADDITIONAL RESOURCES

- Manage RHEL inventory using natural language

3.2. USE CASES AND EXAMPLES

The sample prompts are available to copy and paste, or you can type your own natural-language prompts.

Use case	Sample prompt
Use MCP server for Red Hat Lightspeed as a daily security assistant. Receive data from Red Hat Lightspeed about vulnerability and remediations that require action.	List the top current vulnerabilities across my managed systems and suggest applicable remediations. Returns a list of CVE alerts.
Use MCP server for Red Hat Lightspeed for automated image creation. Prompt Red Hat Lightspeed image builder to generate an optimized, up-to-date RHEL image.	Generate a system image with the latest security patches and recommended packages.
Use MCP server for Red Hat Lightspeed for inventory reconciliation. Ask the server to provide an overview of your active systems.	Provide the list of active hosts and prioritize them based on their advisory status. Returns an overview and a list of all systems, with critical-level advisories listed first.
Use MCP server for Red Hat Lightspeed for compliance monitoring. Use the inventory, vulnerability, and advisor services to track compliance gaps and rank them in order of priority.	Which systems are missing critical remediations or are overdue for updates?
Use MCP server for Red Hat Lightspeed as an incident response assistant. Get a report about systems that require attention.	My production environment has systems failing health checks. Show vulnerabilities and suggest fixes.

Additional resources

- Manage RHEL inventory using natural language
- All available toolsets

3.3. SHARE YOUR FEEDBACK

Now is a great time to test, experiment, and provide feedback when you connect MCP server for Red Hat Lightspeed with your LLMs.

Whether you're exploring automation, enhancing incident processes, or building intelligent dashboards, this preview places powerful Red Hat Lightspeed capabilities at your LLM-driven fingertips.

This release offers early access to powerful MCP-driven workflows with Red Hat Lightspeed. We strongly encourage your feedback — including bug reports, requests for additional toolsets, and enhancement ideas — through the Red Hat Issue Router and by contributing to our GitHub repository. Your input will directly refine and shape the future of MCP server for Red Hat Lightspeed.

Additional resources

- **red-hat-lightspeed-mcp** Contributing Guide
- Red Hat Issue Router
- MCP server for Red Hat Lightspeed GitHub repository